Learning Objectives

Learners will be able to…

- **Explain the concept of AVL trees and describe how they maintain balance to ensure efficient operations.**
- **Calculate the balancing factor of AVL tree nodes and apply rotations to maintain the tree's balanced state.**
- **Perform insertions in AVL trees while preserving the balance through appropriate rotations and adjustments.**
- **Execute deletions in AVL trees and understand the necessary steps to rebalance the tree after removals.**
- **Analyze the time and space complexity of AVL tree operations and appreciate the performance implications of maintaining a balanced tree.**

Introduction to AVL Trees

Balanced Binary Search Trees

As we delve deeper into the world of advanced trees, one of the first balanced binary search trees we encounter is the **AVL Tree**. Named after its inventors Adelson-Velsky and Landis, the AVL tree stands out due to its self-balancing nature, ensuring that the tree remains efficient for operations like insertion, deletion, and lookup.

info

An **AVL Tree** is a binary search tree that maintains its balance through rotations during insertions and deletions. This balance ensures that the tree remains efficient for operations like insertion, deletion, and lookup.

Why AVL Trees?

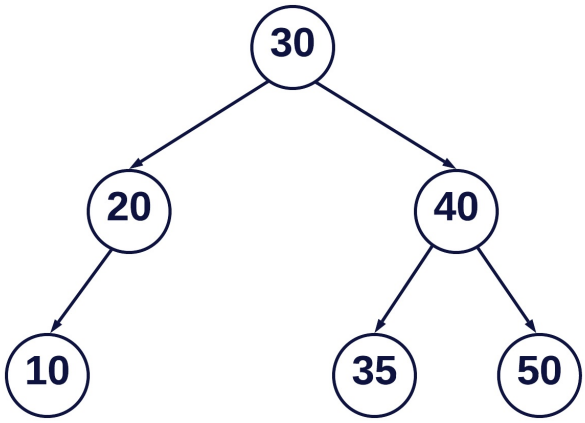
In binary search trees (BST), the time complexity of operations depends on the height of the tree. In the worst-case scenario, a BST can become skewed, leading to operations taking linear time. AVL trees, with their self-balancing property, ensure that the height of the tree is always logarithmic to the number of nodes, guaranteeing efficient operations.

Characteristics of AVL Trees:

- **Balance Factor:** For every node in the tree, the difference between the height of its left and right child is at most 1. This difference, termed as the balance factor, ensures the tree remains balanced. The balance factor is calculated as:
  - *Balance Factor (node) = Height (left subtree) - Height (right subtree)*
- **Rotations:** AVL trees employ four types of rotations to maintain their balance after insertions and deletions:
  - **LL Rotation:** Left rotation around the unbalanced node
  - **RR Rotation:** Right rotation around the unbalanced node
  - **LR Rotation:** Left rotation followed by a right rotation
  - **RL Rotation:** Right rotation followed by a left rotation
    - These rotations adjust the structure of the tree without violating the BST property.
- **Logarithmic Height:** Due to its balancing criteria, the height of an AVL tree is always $O(\log n)$, where n is the number of nodes. This ensures that operations like insertion, deletion, and lookup remain efficient.

Visual Representation of an AVL Tree:

Consider the following AVL tree:



In this tree, every node adheres to the AVL tree property. The balance factor for each node is either 0, 1, or -1, ensuring that the tree remains balanced.

Understanding AVL trees is crucial as they lay the foundation for other self-balancing trees. Their ability to maintain balance ensures that operations are efficient, making them a popular choice in scenarios where the input data might lead to skewed or unbalanced BSTs.

Implementing an AVL Tree

we are going to start building an AVL Tree. They start off in a similar manner to a binary search tree. As we learn more about actions unique to AVL Trees, we will update the code started on this page.

Creating the Node

As usual, we start by creating the `Node` class. Notice that this node has an additional attribute, `height`. In the constructor, set the `value` attribute to the parameter `v`, `height` has the initial value of `1`, while the left and right pointers are `nullptr`.

```
class Node {
public:
    int value, height;
    Node* left;
    Node* right;

    Node(int v) : value(v), height(1), left(nullptr),
        right(nullptr) {}
};
```

Creating the Tree

Next, we are going to create the AVLTree class. This class has only one attribute, the root node. In the constructor, set the root to `nullptr`.

```
class AVLTree {
private:
    Node* root;

public:
    AVLTree() : root(nullptr) {}

};
```

Next, we are going to create some methods to insert a node into the tree. Just as before, we are going to use method overriding. The user-facing method takes a single integer of the value to be added to the tree. Then update root to have the value returned by private method `insert`. The private method takes two parameters, the root node and the value to be inserted.

```
public:
    void insert(int value) {
        root = insert(root, value);
    }
```

For now, we are going to insert a node without taking into account balancing the tree. Start by checking if the tree is empty. If so, create a new node with the given value and return this node.

```
private:
    Node* insert(Node* node, int value) {
        if (node == nullptr)
            return new Node(value);
```

Otherwise, we need to find the proper location for the new value. If the given value is less than the value of the current node, assign the left pointer to `insert(node->left, value)`. This recurses over the left subtree. After finishing recursing over the left subtree, we need to do the same thing to the right subtree `(insert(node->right, value))`. If no changes are to be made, return the node.

```
        if (value < node->value)
                node->left = insert(node->left, value);
            else if (value > node->value)
                node->right = insert(node->right, value);

            return node;
        }
```

So that we can verify that node insertion is working as expected, we are going to create the `preOrder` method that prints the nodes from a pre-order traversal. Just like the `insert` method we are going to have public and private overloaded methods to print the `tree`. The public method does not have any parameters. It calls the private method and passes it `root`.

```
public:
    void preOrder() {
        preOrder(root);
    }
```

Declare the private `preOrder` method which takes a Node object. As long as the node is not `nullptr`, print the value of the node and a space. Recursively call the `preOrder` method twice. The first time with the left subtree, and the second with the right subtree.

```
private:
    void preOrder(Node* node) {
        if (node != nullptr) {
            std::cout << node->value << " ";
            preOrder(node->left);
            preOrder(node->right);
        }
    }
```

Create the `main` method such that you are instantiating an `AVLTree` object. Insert nodes with the values `10` (this will be the root), `20`, `30`, `40`, `50`, and `25`.

```
int main() {
    AVLTree tree;

    // Insert nodes
    tree.insert(10);
    tree.insert(20);
    tree.insert(30);
    tree.insert(40);
    tree.insert(50);
    tree.insert(25);

    // Print tree preorder
    tree.preOrder();

    return 0;
}
```

▼ Code

Your code should look like this:

```
#include <iostream>

class Node {
public:
    int value, height;
    Node* left;
    Node* right;

    Node(int v) : value(v), height(1), left(nullptr),
        right(nullptr) {}
};

class AVLTree {
private:
    Node* root;

    Node* insert(Node* node, int value) {
        if (node == nullptr)
            return new Node(value);

        if (value < node->value)
            node->left = insert(node->left, value);
```

```
        else if (value > node->value)
            node->right = insert(node->right, value);

        return node;
    }

    void preOrder(Node* node) {
        if (node != nullptr) {
            std::cout << node->value << " ";
            preOrder(node->left);
            preOrder(node->right);
        }
    }

public:
    AVLTree() : root(nullptr) {}

    void insert(int value) {
        root = insert(root, value);
    }

    void preOrder() {
        preOrder(root);
    }
};

int main() {
    AVLTree tree;

    // Insert nodes
    tree.insert(10);
    tree.insert(20);
    tree.insert(30);
    tree.insert(40);
    tree.insert(50);
    tree.insert(25);

    // Print tree preorder
    tree.preOrder();

    return 0;
}
```

You should see the following output:

```
10 20 30 25 40 50
```

Balancing Factor & Rotations

Diving deeper into the mechanics of AVL trees, two fundamental concepts stand out: the **balancing factor** and the **rotations**. These are the pillars that ensure the self-balancing nature of AVL Trees, guaranteeing their efficiency.

Balancing Factor

The balancing factor of a node in an AVL tree is the difference between the height of its left subtree and the height of its right subtree. It's a measure that helps us determine if a tree (or a subtree) is balanced or not.

info

Equation:

Balancing Factor (node) = Height (left subtree) - Height (right subtree)

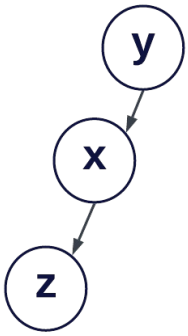
For an AVL tree to be balanced, the balancing factor of every node must be in the range of `-1` to `1`. If the balancing factor goes beyond this range after an insertion or deletion, the tree becomes unbalanced and requires rotations to restore its balance.

Types of Rotations

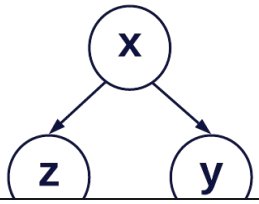
Rotations are the techniques used to restore balance in an AVL tree when it becomes unbalanced. There are four primary rotations:

1. **RR Rotation (Right Rotation):** This rotation is applied when the left subtree of the left child of a node becomes longer than the right subtree. The unbalanced node is rotated to the right.

Before RR Rotation:



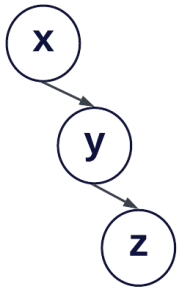
After RR Rotation:



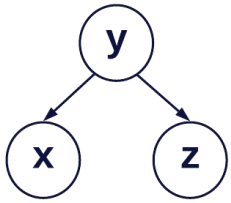
---

2. **LL Rotation (Left Rotation):** This rotation is applied when the right subtree of the right child of a node becomes longer than the left subtree. The unbalanced node is rotated to the left.

Before LL Rotation:



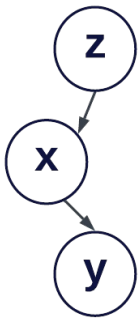
After LL Rotation:



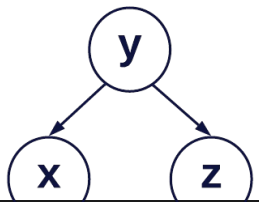
---

3. **LR Rotation (Left-Right Rotation):** This rotation is a combination of a left rotation followed by a right rotation. It's applied when the right subtree of the left child of a node becomes longer than its left subtree.

Before LR Rotation:



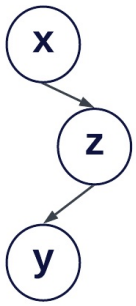
After LR Rotation:



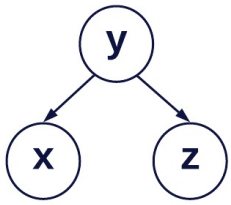
---

4. **RL Rotation (Right-Left Rotation):** This rotation is a combination of a right rotation followed by a left rotation. It's applied when the left subtree of the right child of a node becomes longer than its right subtree.

Before RL Rotation:



After RL Rotation:



Understanding the balancing factor and rotations is crucial for AVL Trees. These mechanisms ensure that the tree remains balanced after every insertion or deletion, guaranteeing logarithmic time complexities for these operations. As we delve deeper into AVL Trees, we'll see how these rotations are applied in practice to maintain the tree's balance.

Implementing Rotations

We are going to create the methods for rotating to the left and to the right. For now, we are not going to worry about left-right and right-left rotations. We are just going to focus on left and right rotations.

Helper Methods

Part of the rotation process is recalculating the height of each node after rotation. To help us do this, we are going to create the `height` method that takes a node pointer. If the node pointer is `nullptr`, then return `-1` as the height. Otherwise, return the `height` attribute of the node.

```
private:
    int height(Node* node) {
        return node ? node->height : -1;
    }
```

We also need a method that updates the height of a node. Create the `updateHeight` method that takes a `Node*` object. Using the `height` method, get the height of the left and right children. Set the height of the current node to the largest height of the children plus `1`.

```
private:
    void updateHeight(Node* node) {
        int leftChildHeight = height(node->left);
        int rightChildHeight = height(node->right);
        node->height = std::max(leftChildHeight,
        rightChildHeight) + 1;
    }
```

Since the rotation methods are going to be `private`, we are going to make a temporary method called `makeRotation` that calls a rotation. We are going to start with a left rotation. **Important**, this method will not be a part of our final code base, we are only using it to verify that our rotations are working.

```
public:
    void makeRotation() {
        root = rotateLeft(root);
    }
```

Left Rotation

Create the `rotateLeft` method for a left rotation. This method takes a `Node*` object. If either the given node or its right subtree is `nullptr`, return the current node. Create the `rightChild` variable that is the right child of the given node. Update the value of the right child to `rightChild->left`, and then `rightChild->left` takes the value of the given node. Update the heights for the given node and `rightChild`. Finally, return `rightChild`.

```
private:
    Node* rotateLeft(Node* node) {
        if (!node || !node->right) {
            return node;
        }

        Node* rightChild = node->right;
        node->right = rightChild->left;
        rightChild->left = node;

        updateHeight(node);
        updateHeight(rightChild);

        return rightChild;
    }
```

Update the `main` method so that three nodes with the values `10`, `20`, and `30` are added to the tree in that order. Rotate the tree to the left and print the tree with a pre-order traversal.

```
int main() {
    AVLTree tree;

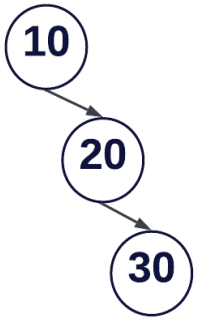
    // Insert nodes
    tree.insert(10);
    tree.insert(20);
    tree.insert(30);

    // Rotate and print tree
    std::cout << "Before rotation: ";
    tree.preOrder();
    std::cout << "\nAfter rotation: ";
    tree.makeRotation();
    tree.preOrder();

    return 0;
}
```

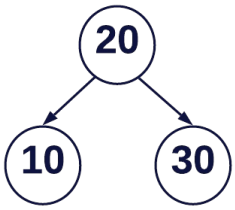
Inserting those nodes in that order would create an unbalanced tree that looks like this:

Before Left Rotation:



Rotating the tree to the left should alter the tree so that it now looks like this:

After Left Rotation:



▼ **Code**

Your code should look like this:

```
class Node {
public:
    int value, height;
    Node *left, *right;

    Node(int v) : value(v), height(0), left(nullptr),
        right(nullptr) {}
};

class AVLTree {
private:
    Node *root;

    Node* insert(Node* node, int value) {
        if (node == nullptr)
            return new Node(value);

        if (value < node->value)
            node->left = insert(node->left, value);
        else if (value > node->value)
```

```
            node->right = insert(node->right, value);

        return node;
    }

    void preOrder(Node* node) {
        if (node != nullptr) {
            std::cout << node->value << " ";
            preOrder(node->left);
            preOrder(node->right);
        }
    }

    int height(Node* node) {
        return node != nullptr ? node->height : -1;
    }

    void updateHeight(Node* node) {
        int leftChildHeight = height(node->left);
        int rightChildHeight = height(node->right);
        node->height = std::max(leftChildHeight,
        rightChildHeight) + 1;
    }

    Node* rotateLeft(Node* node) {
        if (!node || !node->right) {
            return node;
        }

        Node* rightChild = node->right;
        node->right = rightChild->left;
        rightChild->left = node;

        updateHeight(node);
        updateHeight(rightChild);

        return rightChild;
    }

public:
    AVLTree() : root(nullptr) {}

    void insert(int value) {
        root = insert(root, value);
    }

    void preOrder() {
        preOrder(root);
    }

    void makeRotation() {
        root = rotateLeft(root);
    }
};

int main() {
    AVLTree tree;
```

```
    // Insert nodes
    tree.insert(10);
    tree.insert(20);
    tree.insert(30);

    // Rotate and print tree
    std::cout << "Before rotation: ";
    tree.preOrder();
    std::cout << "\nAfter rotation: ";
    tree.makeRotation();
    tree.preOrder();

    return 0;
}
```

Run the program. If everything is working as expected, we should see the following output from the pre-order traversals:

```
Before rotation: 10 20 30
After rotation: 20 10 30
```

Right Rotation

A right rotation works in a similar manner. Create the `rotateRight` method for a right rotation. This method takes a `Node*` object. If either the given node or its left subtree is nullptr, return the current node. Create the `leftChild` variable and set its value to the left child of the given node. Update the value of the left child to `leftChild->right`, and then `leftChild->right` takes the value of the given node. Update the heights for the given node and `leftChild`. Finally, return `leftChild`.

```
    Node* rotateRight(Node* node) {
        if (!node || !node->left) {
            return node;
        }

        Node* leftChild = node->left;
        node->left = leftChild->right;
        leftChild->right = node;

        updateHeight(node);
        updateHeight(leftChild);

        return leftChild;
    }
```

Update the `makeRotation` method so that it now calls `rotateRight`.

```
    void makeRotation() {
        root = rotateRight(root);
    }
```

Then update the `main` method so the AVL tree has nodes with the values `30`, `20`, and `10` (in that order). Print the tree before and after the rotation.

```
int main() {
    AVLTree tree;

    // Insert nodes
    tree.insert(30);
    tree.insert(20);
    tree.insert(10);

    // Rotate and print tree
    std::cout << "Before rotation: ";
    tree.preOrder();
    std::cout << "\nAfter rotation: ";
    tree.makeRotation();
    tree.preOrder();

    return 0;
}
```

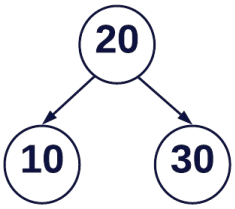
Inserting the nodes in the order specified above will create a tree that looks like this:

Before Right Rotation:



Rotating the tree to the right should alter the tree so that it now looks like this:

After Right Rotation:



▼ **Code**

Your code should look like this:

```
class Node {
public:
    int value, height;
    Node *left, *right;

    Node(int v) : value(v), height(0), left(nullptr),
        right(nullptr) {}
};

class AVLTree {
private:
    Node *root;

    Node* insert(Node* node, int value) {
        if (node == nullptr)
            return new Node(value);

        if (value < node->value)
            node->left = insert(node->left, value);
        else if (value > node->value)
            node->right = insert(node->right, value);

        return node;
    }

    void preOrder(Node* node) {
        if (node != nullptr) {
            std::cout << node->value << " ";
            preOrder(node->left);
            preOrder(node->right);
        }
    }

    int height(Node* node) {
        return node != nullptr ? node->height : -1;
    }

    void updateHeight(Node* node) {
        int leftChildHeight = height(node->left);
        int rightChildHeight = height(node->right);
```

```
        node->height = std::max(leftChildHeight,
        rightChildHeight) + 1;
    }

    Node* rotateLeft(Node* node) {
        if (!node || !node->right) {
            return node;
        }

        Node* rightChild = node->right;
        node->right = rightChild->left;
        rightChild->left = node;

        updateHeight(node);
        updateHeight(rightChild);

        return rightChild;
    }

  Node* rotateRight(Node* node) {
        if (!node || !node->left) {
            return node;
        }

        Node* leftChild = node->left;
        node->left = leftChild->right;
        leftChild->right = node;

        updateHeight(node);
        updateHeight(leftChild);

        return leftChild;
  }

public:
    AVLTree() : root(nullptr) {}

    void insert(int value) {
        root = insert(root, value);
    }

    void preOrder() {
        preOrder(root);
    }

    void makeRotation() {
        root = rotateRight(root);
    }
};

int main() {
    AVLTree tree;

    // Insert nodes
    tree.insert(30);
    tree.insert(20);
    tree.insert(10);
```

```
    // Rotate and print tree
    std::cout << "Before rotation: ";
    tree.preOrder();
    std::cout << "\nAfter rotation: ";
    tree.makeRotation();
    tree.preOrder();

    return 0;
}
```

Run the program. If everything is working as expected, we should see the following output from the pre-order traversals:

```
Before rotation: 30 20 10
After rotation: 20 10 30
```

Insertion in AVL Trees

Inserting and Balancing

Insertion in AVL Trees follows a similar process to that of a regular Binary Search Tree (BST), with the added step of ensuring that the tree remains balanced after each insertion. Let's delve into the process:

1. Standard BST Insertion:

Insert the new node as you would in a regular BST. If the node to be inserted has a value less than the current node, it goes to the left; if greater, it goes to the right.

2. Update Height:

After insertion, traverse back from the inserted node up to the root. Update the height of each node. The height of a node is the maximum of the heights of its left and right children, plus one.

3. Get the Balance Factor:

For each visited node, calculate its balance factor. The balance factor of a node is the height of its left child minus the height of its right child. A balance factor can be -1, 0, or 1 for a balanced tree. If the balance factor is outside this range (less than -1 or greater than 1), it indicates that the tree is unbalanced at that node.

4. Perform Rotations:

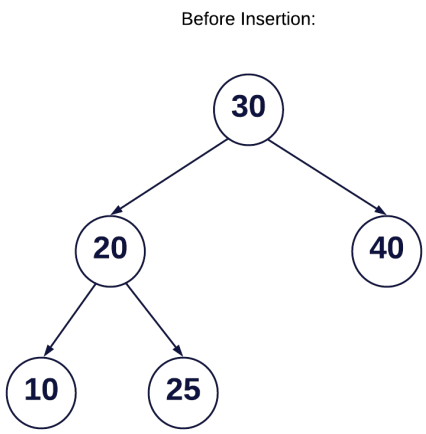
Depending on the balance factor and the side where the new node was inserted, decide which rotation to apply:

- **RR Imbalance (Right Rotation)**: If the balance factor of the current node is > 1 and the key to be inserted is smaller than the key of the left child of the node, then this is an LL case. Perform a right rotation.
- **LL Imbalance (Left Rotation)**: If the balance factor of the current node is < -1 and the key to be inserted is greater than the key of the right child of the node, then this is an RR case. Perform a left rotation.
- **LR Imbalance (Left-Right Rotation)**: If the balance factor is > 1 and the key to be inserted is greater than the key of the left child of the node, then this is an LR case. Perform a left rotation on the left child, followed by a right rotation on the current node.

- **RL Imbalance (Right-Left Rotation)**: If the balance factor is < -1 and the key to be inserted is smaller than the key of the right child of the node, then this is an RL case. Perform a right rotation on the right child, followed by a left rotation on the current node.

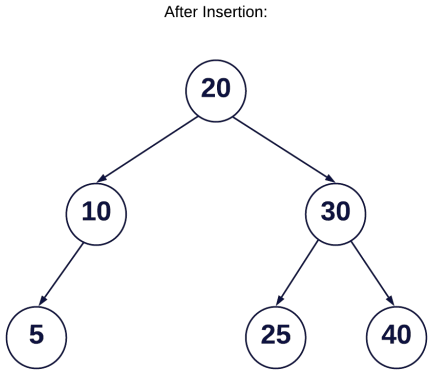
Example:

Consider the following AVL Tree:



If we insert a node with value `5`, it will go to the left of `10`, causing an LL imbalance at node `30`. To balance the tree, we perform a right rotation around `30`.

Resulting tree:



Conclusion:

Insertion in AVL Trees ensures that the tree remains approximately balanced, which guarantees logarithmic time bounds for insert, delete, and find operations. The rotations are the key to maintaining this balance. Understanding the different types of imbalances and how to correct them with rotations is crucial for implementing and working with AVL Trees.

Implementing Insertion

info

Remove `makeRotation` Method

On a previous page, we created the `makeRotation` method to force a rotation on our tree. We are going incorporate this functionality in the `insert` method. Remove `makeRotation` as it is no longer needed.

```
    void makeRotation() {
        root = rotateRight(root);
    }
```

Calculating the Balance Factor

Before we update our insert method, we need to be able to calculate the balance factor. This will help us decide what kind of rotation is needed. Create the `balanceFactor` method which takes a node. Return the difference between the height of the right subtree and the height of the left subtree.

```
    int balanceFactor(Node* node) {
        return height(node->left) - height(node->right);
    }
```

Inserting a Node

We need to update our current insert method. Add an else clause to the conditional which returns the unchanged node when trying to insert a duplicate value. Then update the height of the node. Finally, return a re-balanced node.

```
Node* insert(Node* node, int value) {

    if (node == nullptr)
        return (new Node(value));

    if (value < node->value)
        node->left = insert(node->left, value);
    else if (value > node->value)
        node->right = insert(node->right, value);
    else
        return node;

    updateHeight(node);

    return rebalance(node);
}
```

Balancing the Tree

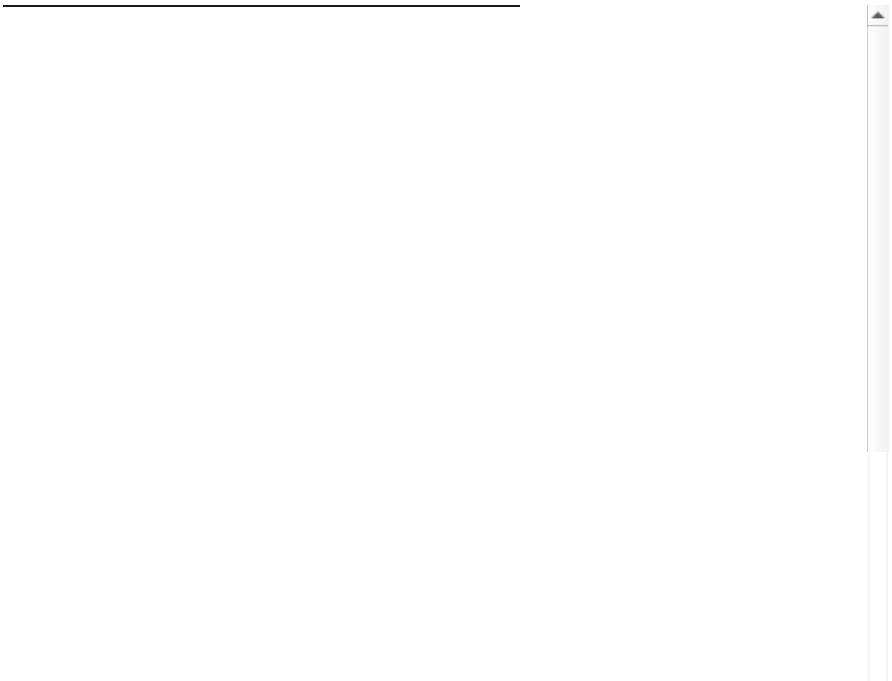
This the most important operation of an AVL tree. If it is not properly balanced, then you lose all of the benefits with regards to time complexity. There are four different kinds of rotations that can be performed when balancing a tree: left rotation, right rotation, left-right rotation, and right-left rotation. Let's start by creating the `rebalance` method which takes a Node object. The first thing to do is calculate the balance factor. We will then look at each of the different rotation cases.

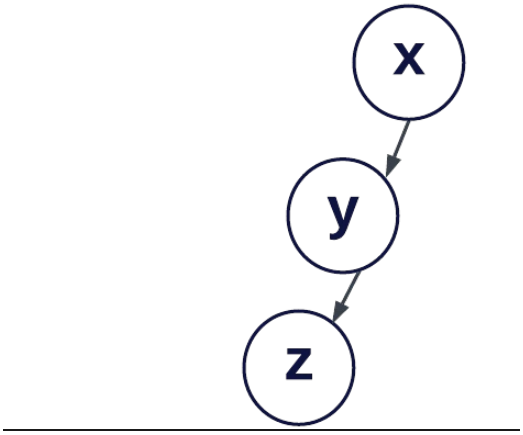
```
Node* rebalance(Node* node) {
    int balanceFactor =this->balanceFactor(node);

}
```

Case 1 - Right Rotation

We are going to start with a balance factor that is greater than `1`. This means the tree is left-heavy. The tree would look something like this:





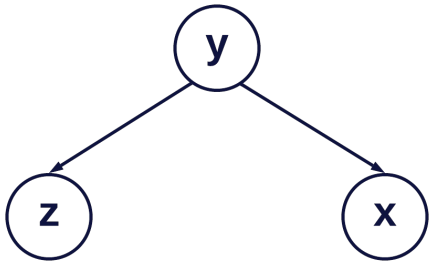
Balancing is rather simple. We can define this situation in code when the balance factor of the given node (x in the image above) is greater than one and the balance factor of the left child (y in the image above) is greater than zero. In this case, perform a simple right rotation.

```
    Node* rebalance(Node* node) {
        int balanceFactor = this->balanceFactor(node);

        // Left-heavy
        if (balanceFactor > 1) {
            if (this->balanceFactor(node->left) > 0) {
                // Case 1: Left-Left
                node = rotateRight(node);
            }
        }

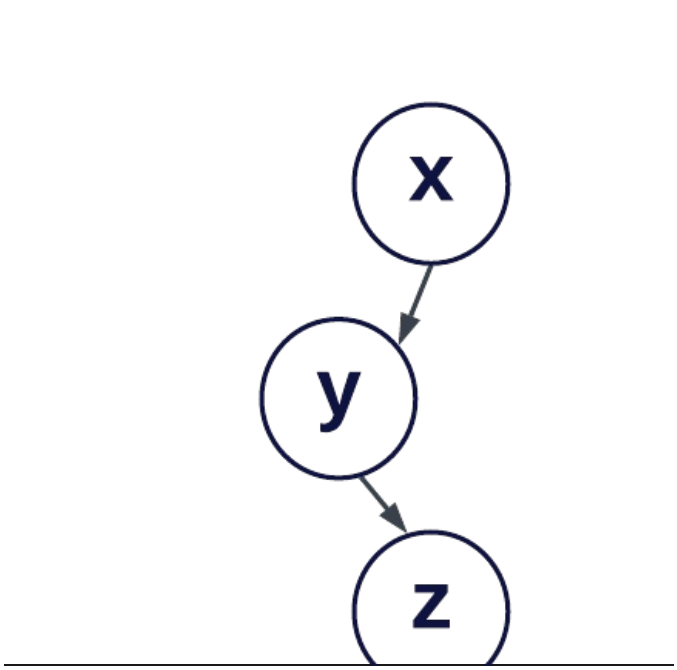
    }
```

The above code would produce a tree that looks like this:

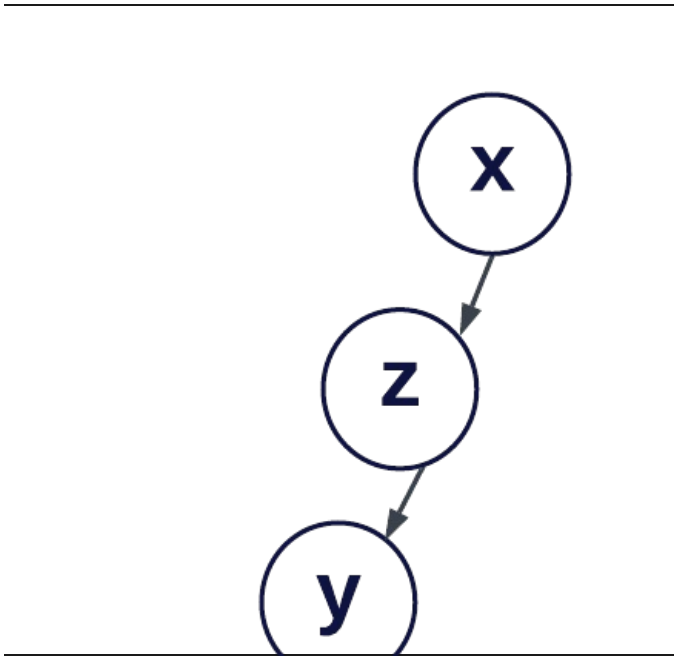


Case 2 - Left-Right Rotation

However, just because a tree is left-heavy does not mean you will always perform a left rotation. You may encounter a tree that looks like this:



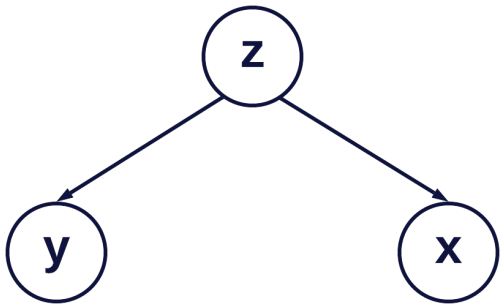
This is still left-heavy, but node `z` is the right child of node `y`. In this case, we are first going to perform a left rotation. Our tree would then looks like this:



We have seen this structure before, so all we need to do is perform a right rotation to balance the tree. We can express this in code by adding an else statement for when the balance factor of the left child is less than zero. In this case, perform a left rotation on the left child. Then perform a right rotation on the node.

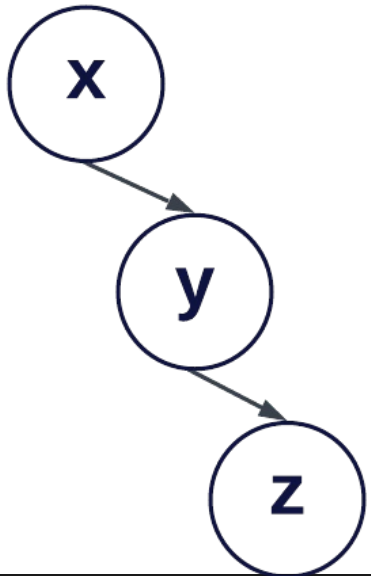
```
        // Left-heavy
        if (balanceFactor > 1) {
            if (this->balanceFactor(node->left) > 0) {
                // Case 1: Left-Left
                node = rotateRight(node);
            } else {
                // Case 2: Left-Right
                node->left = rotateLeft(node->left);
                node = rotateRight(node);
            }
        }
```

The above code would produce a tree that looks like this:



Case 3 - Left Rotation

When the balance factor is less than `-1`, the tree is said to be right-heavy. The tree would look something like this:

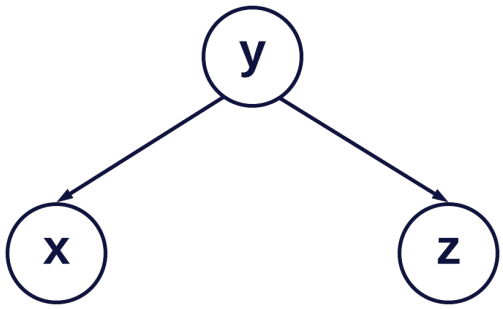


We can express this scenario in code by adding an else-if statement to the `rebalance` method that checks if the balance factor of the parent (x in the image above) is less than -1. In addition, the balance factor of the right child (y in the image above) is less than zero. In this case, perform a left rotation.

```
        // Right-heavy
        else if (balanceFactor < -1) {
            if (this->balanceFactor(node->right) < 0) {
                // Case 3: Right-Right
                node = rotateLeft(node);
            }

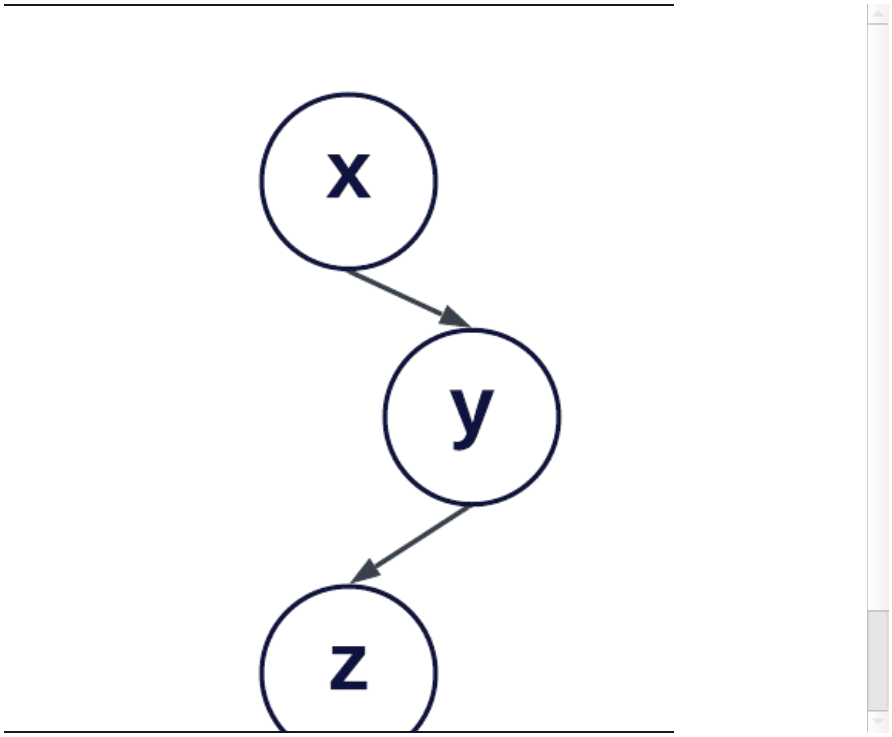
        }
```

The above code would produce a tree that looks like this:

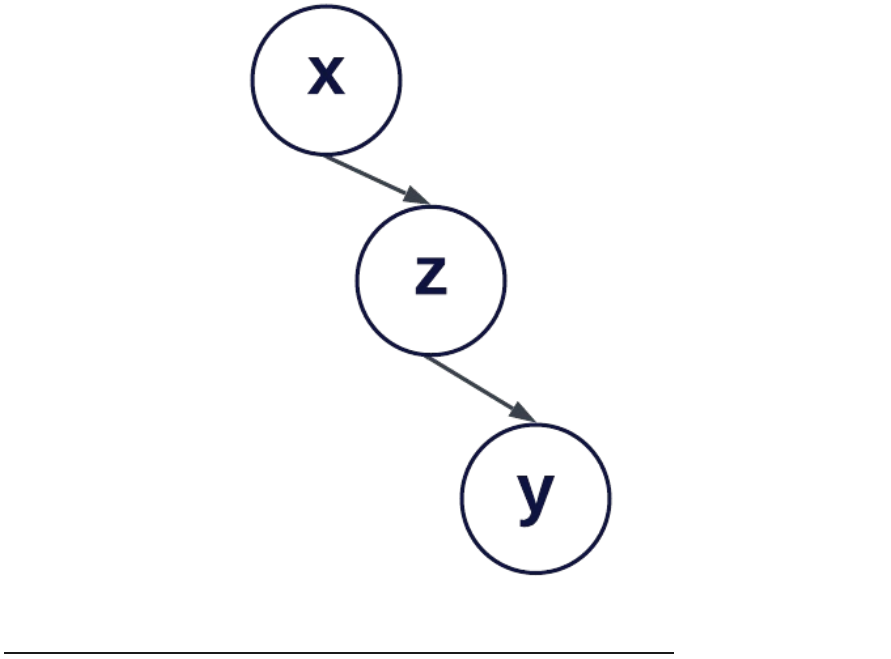


Case 4 - Right-Left Rotation

The final case is when a tree is right-heavy but a simple left rotation will not work to re-balance the tree.



Instead, we can perform a right rotation on node `y`, which would give a tree that looks like this:

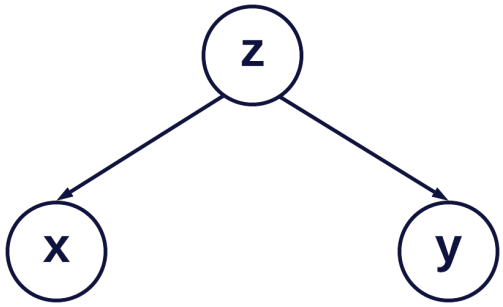


Now a left rotation will balance the tree. We can express this scenario by adding an else statement to handle cases where the balance factor of the right child (node `y` in the first image of Case 4) is greater than zero. First perform a right rotation on the right child (node `y` in the first image of Case 4), and then perform a left rotation on the parent (node `z` in the second image of Case 4). Finally return the node.

```
        // Right-heavy
        else if (balanceFactor < -1) {
            if (this->balanceFactor(node->right) < 0) {
                // Case 3: Right-Right
                node = rotateLeft(node);
            } else {
                // Case 4: Right-Left
                node->right = rotateRight(node->right);
                node = rotateLeft(node);
            }
        }

        return node;
```

The above code would produce a tree that looks like this:



Using the example from the previous page, update the main method to contain nodes with the values `30`, `20`, `40`, `10`, and `25`. Print the tree. Then add a node with the value of 5 and print the tree again.

```
int main() {
    AVLTree tree;

    // Insert nodes
    tree.insert(30);
    tree.insert(20);
    tree.insert(40);
    tree.insert(10);
    tree.insert(25);

    // Insert and print tree
    std::cout << "Before insertion: ";
    tree.preOrder();
    std::cout << "\nAfter insertion: ";
    tree.insert(5);
    tree.preOrder();

    return 0;
}
```

▼ **Code**

Your code should look like this:

```
class Node {
public:
    int value, height;
    Node *left, *right;

    Node(int v) : value(v), height(0), left(nullptr),
        right(nullptr) {}
};

class AVLTree {
private:
    Node *root;
    int balanceFactor(Node* node) {
        return height(node->left) - height(node->right);
    }

    Node* rebalance(Node* node) {
        int balanceFactor = this->balanceFactor(node);

      // Left-heavy
        if (balanceFactor > 1) {
            if (this->balanceFactor(node->left) > 0) {
                // Case 1: Left-Left
                node = rotateRight(node);
            } else {
                // Case 2: Left-Right
                node->left = rotateLeft(node->left);
                node = rotateRight(node);
            }
        }
                // Right-heavy
        else if (balanceFactor < -1) {
```

```
        if (this -> balanceFactor(node->right) < 0) {
            // Case 3: Right-Right
            node = rotateLeft(node);
        } else {
            // Case 4: Right-Left
            node->right = rotateRight(node->right);
            node = rotateLeft(node);
        }
    }

    return node;

}


Node* insert(Node* node, int value) {
    if (node == nullptr)
        return (new Node(value));

    if (value < node->value)
        node->left = insert(node->left, value);
    else if (value > node->value)
        node->right = insert(node->right, value);
    else
        return node;

    updateHeight(node);

    return rebalance(node);
}

void preOrder(Node* node) {
    if (node != nullptr) {
        std::cout << node->value << " ";
        preOrder(node->left);
        preOrder(node->right);
    }
}

int height(Node* node) {
    return node != nullptr ? node->height : -1;
}

void updateHeight(Node* node) {
    int leftChildHeight = height(node->left);
    int rightChildHeight = height(node->right);
    node->height = std::max(leftChildHeight,
    rightChildHeight) + 1;
}
Node* rotateLeft(Node* node) {
    if (!node || !node->right) {
        return node;
    }

    Node* rightChild = node->right;
    node->right = rightChild->left;
    rightChild->left = node;
```

```
        updateHeight(node);
        updateHeight(rightChild);

        return rightChild;
    }

    Node* rotateRight(Node* node) {
        if (!node || !node->left) {
            return node;
        }

        Node* leftChild = node->left;
        node->left = leftChild->right;
        leftChild->right = node;

        updateHeight(node);
        updateHeight(leftChild);

        return leftChild;
    }


public:
    AVLTree() : root(nullptr) {}

    void insert(int value) {
        root = insert(root, value);
    }

    void preOrder() {
        preOrder(root);
    }
};

int main() {
    AVLTree tree;

    // Insert nodes
    tree.insert(30);
    tree.insert(20);
    tree.insert(40);
    tree.insert(10);
    tree.insert(25);

    // Insert and print tree
    std::cout << "Before insertion: ";
    tree.preOrder();
    std::cout << "\nAfter insertion: ";
    tree.insert(5);
    tree.preOrder();

    return 0;
}
```

Run the program. If everything is working as expected, we should see the following output from the pre-order traversals:

```
Before insertion: 30 20 10 25 40
After insertion: 20 10 5 30 25 40
```

Deletion in AVL Trees

Removing and Rebalancing

Deletion in AVL Trees is slightly more complex than insertion because, after a node is removed, the tree might become unbalanced at more than one point. The process involves removing the node as in a regular Binary Search Tree (BST) and then ensuring the tree remains balanced.

1. Standard BST Deletion:

Remove the node as you would in a regular BST. There are three cases to consider:

- **Node with only one child or no child**: Simply remove the node.
- **Node with two children**: Find the in-order predecessor (maximum value node in the left subtree) or the in-order successor (minimum value node in the right subtree), copy its value to the node, and then recursively delete the in-order predecessor or successor.

2. Update Height:

After deletion, traverse back from the deleted node up to the root. Update the height of each node. The height of a node is the maximum of the heights of its left and right children, plus one.

3. Get the Balance Factor:

For each visited node, calculate its balance factor. If the balance factor is outside the range of -1 to 1, it indicates that the tree is unbalanced at that node.

4. Perform Rotations:

Depending on the balance factor and the balance factors of the children, decide which rotation to apply:

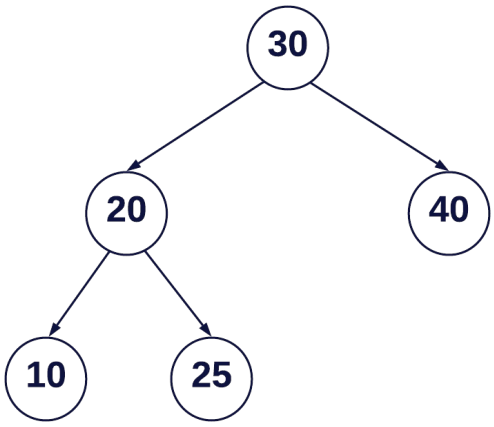
- **LL Imbalance (Right Rotation)**: If the balance factor of the current node is > 1 and the balance factor of the left child is >= 0, then this is an LL case. Perform a right rotation.

- **RR Imbalance (Left Rotation)**: If the balance factor of the current node is < -1 and the balance factor of the right child is <= 0, then this is an RR case. Perform a left rotation.

- **LR Imbalance (Left-Right Rotation)**: If the balance factor is > 1 and the balance factor of the left child is < 0, then this is an LR case. Perform a left rotation on the left child, followed by a right rotation on the current node.
- **RL Imbalance (Right-Left Rotation)**: If the balance factor is < -1 and the balance factor of the right child is > 0, then this is an RL case. Perform a right rotation on the right child, followed by a left rotation on the current node.

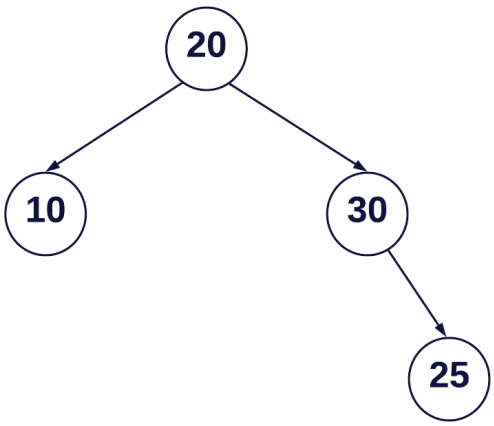
Example:

Consider the following AVL Tree:



If we delete the node with value `40`, the tree becomes unbalanced with an LL imbalance at node `30`. To balance the tree, we perform a right rotation around `30`.

Resulting tree:



Conclusion:

Deletion in AVL Trees ensures that the tree remains approximately balanced after nodes are removed. This balance guarantees logarithmic time bounds for insert, delete, and find operations. The rotations, as in insertion, are crucial for maintaining this balance. Understanding how to balance the tree after deletions is essential for anyone working with AVL Trees.

Implementing Deletion

BST Deletion

Deleting a node from an AVL tree starts off like a deletion from a BST. To do this, we need a helper method that finds the in-order successor, i.e. the node with the minimum value in a subtree.

```
private:
    Node* minValueNode(Node* node) {
        Node* current = node;

        while (current->left != nullptr)
            current = current->left;

        return current;
    }
```

We are going to use overloaded methods for the initial method call to `delete` a node. The public method takes an integer value to be deleted and calls the private method that takes root and the value.

```
public:
    void deleteValue(int value) {
        root = deleteNode(root, value);
    }
```

When deleting in a BST, return `root` without any changes should the tree be empty. Then recursively move down the left or right subtrees until you find the node to be deleted.

```
private:
    Node* deleteNode(Node* root, int value) {
        if (root == nullptr) {
            return root;
        }

        if (value < root->value) {
            root->left = deleteNode(root->left, value);
        }
        else if (value > root->value) {
            root->right = deleteNode(root->right, value);
        } else {




        }


    }
```

Once we find the node to `delete`, we need to figure out if it has no children, one child, or two children. Start by determining if either the left or right child is `nullptr`. This will handle cases where the node has one or no children. Create the `Node` object `temp` and set it to `nullptr`. If there is no left child, set `temp` to the right child. Otherwise set temp to the left child. If `temp` is still `nullptr` (there are no children), return `nullptr`. Otherwise retur `temp`.

```
        // Node with only one child or no child
        if (root->left == nullptr || root->right == nullptr) {
            Node* temp = nullptr;
            if (root->left == nullptr)
                temp = root->right;
            else
                temp = root->left;

            // No child
            if (temp == nullptr) {
                return nullptr;
            } else {
                // One child
                return temp;
            }
        } else {

        }
```

If the `node` to delete has two children, create the `Node` object temp and set it to the minimum value in the right subtree. Copy this value into the node we want to delete. Then recursively call `deleteNode` with the right subtree and the value of the in-order successor.

```
            // Node with two children, find in-order
        successor
            Node* temp = minValueNode(root->right);

            // Copy the in-order successor's value
            root->value = temp->value;

            // Delete the in-order successor
            root->right = deleteNode(root->right, temp-
        >value);
```

Balance the Tree

Since we already created the `updateHeight` and `rebalance` methods, the hard work is already done. We just need to add a method call for `updateHeight` and return the `rebalance` method at the end of the `deleteNode` method.

```
    Node* deleteNode(Node* root, int value) {
        if (root == nullptr) {
            return root;
        }

        if (value < root->value) {
            root->left = deleteNode(root->left, value);
        }
        else if (value > root->value) {
            root->right = deleteNode(root->right, value);
        } else {
            // Node with only one child or no child
            if (root->left == nullptr || root->right == nullptr)
        {
                Node* temp = nullptr;
                if (root->left == nullptr)
                    temp = root->right;
                else
                    temp = root->left;

                // No child
                if (temp == nullptr) {
                    return nullptr;
                } else {
                    // One child
                    return temp;
                }
            } else {
                // Node with two children, find in-order
        successor
                Node* temp = minValueNode(root->right);

                // Copy the in-order successor's value
                root->value = temp->value;

                // Delete the in-order successor
                root->right = deleteNode(root->right, temp-
        >value);
            }
        }

        updateHeight(root);

        return rebalance(root);
    }
```

Update the main method to add the nodes `9`, `5`, `10`, `0`, `6`, `11`, `1`, and `2`. Print the tree. Remove the node with the value `10` and print the tree again.

```
int main() {
    AVLTree tree;

    // Insert nodes
    tree.insert(9);
    tree.insert(5);
    tree.insert(10);
    tree.insert(0);
    tree.insert(6);
    tree.insert(11);
    tree.insert(1);
    tree.insert(2);

    // Delete and print tree
    std::cout << "Before deletion: ";
    tree.preOrder();
    std::cout << "\nAfter deletion: ";
    tree.deleteValue(10);
    tree.preOrder();
}
```

▼ **Code**

Your code should look like this:

```
class Node {
public:
    int value, height;
    Node *left, *right;

    Node(int v) : value(v), height(0), left(nullptr),
        right(nullptr) {}
};

class AVLTree {
private:
    Node *root;

    int balanceFactor(Node* node) {
        return height(node->left) - height(node->right);
    }

    Node* minValueNode(Node* node) {
        Node* current = node;

        while (current->left != nullptr)
            current = current->left;

        return current;
    }

    Node* deleteNode(Node* root, int value) {
        if (root == nullptr) {
            return root;
        }
```

```
        if (value < root->value) {
            root->left = deleteNode(root->left, value);
        }
        else if (value > root->value) {
            root->right = deleteNode(root->right, value);
        } else {
            if (root->left == nullptr || root->right == nullptr)
        {
                Node* temp = root->left ? root->left : root-
        >right;
                if (temp == nullptr) {
                    return nullptr;
                } else {
                    return temp;
                }
            } else {
                Node* temp = minValueNode(root->right);
                root->value = temp->value;
                root->right = deleteNode(root->right, temp-
        >value);
            }
        }

        updateHeight(root);
        return rebalance(root);
    }

    Node* rebalance(Node* node) {
        int balanceFactor = this->balanceFactor(node);

      // Left-heavy
        if (balanceFactor > 1) {
            if (this->balanceFactor(node->left) > 0) {
                // Case 1: Left-Left
                node = rotateRight(node);
            } else {
                // Case 2: Left-Right
                node->left = rotateLeft(node->left);
                node = rotateRight(node);
            }
        }
                // Right-heavy
        else if (balanceFactor < -1) {
            if (this -> balanceFactor(node->right) < 0) {
                // Case 3: Right-Right
                node = rotateLeft(node);
            } else {
                // Case 4: Right-Left
                node->right = rotateRight(node->right);
                node = rotateLeft(node);
            }
        }

        return node;

    }

    Node* insert(Node* node, int value) {
```

```
    if (node == nullptr)
        return (new Node(value));

    if (value < node->value)
        node->left = insert(node->left, value);
    else if (value > node->value)
        node->right = insert(node->right, value);
    else
        return node;

    updateHeight(node);

    return rebalance(node);
}

void preOrder(Node* node) {
    if (node != nullptr) {
        std::cout << node->value << " ";
        preOrder(node->left);
        preOrder(node->right);
    }
}

int height(Node* node) {
    return node != nullptr ? node->height : -1;
}

void updateHeight(Node* node) {
    int leftChildHeight = height(node->left);
    int rightChildHeight = height(node->right);
    node->height = std::max(leftChildHeight,
    rightChildHeight) + 1;
}

Node* rotateLeft(Node* node) {
    if (!node || !node->right) {
        return node;
    }

    Node* rightChild = node->right;
    node->right = rightChild->left;
    rightChild->left = node;

    updateHeight(node);
    updateHeight(rightChild);

    return rightChild;
}

Node* rotateRight(Node* node) {
    if (!node || !node->left) {
        return node;
    }

    Node* leftChild = node->left;
    node->left = leftChild->right;
    leftChild->right = node;
```

```
        updateHeight(node);
        updateHeight(leftChild);

        return leftChild;
    }


public:
    AVLTree() : root(nullptr) {}

    void insert(int value) {
        root = insert(root, value);
    }

    void preOrder() {
        preOrder(root);
    }

    void makeRotation() {
        root = rotateRight(root);
    }

    void deleteValue(int value) {
        root = deleteNode(root, value);
    }

};

int main() {
    AVLTree tree;

    tree.insert(9);
    tree.insert(5);
    tree.insert(10);
    tree.insert(0);
    tree.insert(6);
    tree.insert(11);
    tree.insert(1);
    tree.insert(2);

    std::cout << "Before deletion: ";
    tree.preOrder();
    std::cout << "\nAfter deletion: ";
    tree.deleteValue(10);
    tree.preOrder();
}
```

Run the program. If everything is working as expected, we should see the following output from the pre-order traversals:

```
Before deletion: 9 5 1 0 2 6 10 11
After deletion: 5 1 0 2 9 6 11
```

Time and Space Complexity of AVL Trees

Analyzing AVL Tree Operations

Understanding the time and space complexities of AVL Trees is crucial for evaluating their efficiency in various applications. AVL Trees, being a type of self-balancing binary search tree, offer improved performance over regular binary search trees, especially in scenarios where the input might be sorted or nearly sorted.

Time Complexity

- **Insertion**: The insertion in an AVL tree ensures that after every insertion, the tree remains balanced. Therefore, the height of the tree is always logarithmic to the number of nodes, i.e., $O(logn)$. Thus, the time complexity for insertion is $O(logn)$.
- **Deletion**: Similar to insertion, even after deletions, the AVL tree remains balanced. The time complexity for deletion is also $O(logn)$.
- **Search (or Lookup)**: Since AVL Trees are a type of binary search tree and they remain balanced, the time taken to search for an element is $O(logn)$.

Space Complexity

- **Storage for Nodes**: The space required for storing n nodes is $O(n)$.
- **Height Information**: For each node, in addition to the usual information (key, left child, right child), we also need to store the height of the node (or the balance factor). This adds a constant amount of space for each node, so it does not change the linear relationship. Thus, the space complexity remains $O(n)$.
- **Auxiliary Space during Rotations**: During rotations (left, right, left-right, right-left), a constant amount of extra space is used for temporary variables. This does not depend on the number of nodes. Hence, the auxiliary space complexity is $O(1)$.

Balancing Factor

One of the reasons AVL Trees can guarantee logarithmic heights is the balancing factor. The balancing factor ensures that the height difference between the left and right subtrees of any node is no more than 1. This

property ensures that the tree remains approximately balanced, leading to the logarithmic time complexities mentioned above.

Example

Consider an AVL Tree with 1 million nodes. In a worst-case scenario, a regular binary search tree might have a height of 1 million (if it becomes a skewed tree). However, an AVL Tree will always have a height of around 20 (because 2^{20} is $1,048,576$). This drastic reduction in height leads to much faster search/insert/delete operations.

Conclusion

AVL Trees, with their self-balancing properties, offer consistent and efficient performance for dynamic set operations like search, insert, and delete. The time complexities are logarithmic, making them suitable for large datasets. The space complexity remains linear with an added constant factor for height information. Understanding these complexities is essential for making informed decisions when choosing data structures for specific applications.

Formative Assessment 1

Formative Assessment 2